Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Icml*, 2010.

Behnam Neyshabur, Hanie Sedghi, and Chiyuan Zhang. What is being transferred in transfer learning? In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 512–523. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper/2020/file/0607f4c705595b911a4f3e7a127b44e0-Paper.pdf.

Anh Nguyen, Jason Yosinski, and Jeff Clune. *Understanding Neural Networks via Feature Visualization: A Survey*, pp. 55–76. Springer International Publishing, Cham, 2019. ISBN 978-3-030-28954-6. doi: 10.1007/978-3-030-28954-6_4. URL https://doi.org/10.1007/978-3-030-28954-6_4.

Thao Nguyen, Maithra Raghu, and Simon Kornblith. Do wide and deep networks learn the same things? uncovering how neural network representations vary with width and depth. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=KJNcAkY8tY4.

Thao Nguyen, Maithra Raghu, and Simon Kornblith. On the origins of the block structure phenomenon in neural network representations. *arXiv preprint arXiv:2202.07184*, 2022.

Edouard Oyallon. Building a regular decision boundary with deep networks. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1886–1894, 2017. doi: 10.1109/CVPR.2017.204.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, Peter J Liu, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(140):1–67, 2020.

Maithra Raghu, Justin Gilmer, Jason Yosinski, and Jascha Sohl-Dickstein. Svcca: Singular vector canonical correlation analysis for deep learning dynamics and interpretability. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper/2017/file/dc6a7e655d7e5840e66733e9ee67cc69-Paper.pdf.

Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in Neural Information Processing Systems*, 34, 2021.

Vinay Venkatesh Ramasesh, Ethan Dyer, and Maithra Raghu. Anatomy of catastrophic forgetting: Hidden representations and task semantics. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=LhY8QdUGSuw.

Stefano Recanatesi, Matthew Farrell, Guillaume Lajoie, Sophie Deneve, Mattia Rigotti, and Eric Shea-Brown. Predictive learning as a network mechanism for extracting low-dimensional latent space representations. *Nature Communications*, 12(1), March 2021. doi: 10.1038/s41467-021-21696-1. URL https://doi.org/10.1038/s41467-021-21696-1.

Le Song, Alex Smola, Arthur Gretton, Justin Bedo, and Karsten Borgwardt. Feature selection via dependence maximization. *Journal of Machine Learning Research*, 13(5), 2012.

David Sussillo and Omri Barak. Opening the Black Box: Low-Dimensional Dynamics in High-Dimensional Recurrent Neural Networks. *Neural Computation*, 25(3):626–649, 03 2013. ISSN 0899-7667. doi: 10.1162/NECO_a_00409. URL https://doi.org/10.1162/NECO_a_00409.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Bastiaan S Veeling, Jasper Linmans, Jim Winkens, Taco Cohen, and Max Welling. Rotation equivariant CNNs for digital pathology. *arXiv preprint arXiv:1806.03962*, June 2018.

Vikas Verma, Alex Lamb, Christopher Beckham, Amir Najafi, Ioannis Mitliagkas, David Lopez-Paz, and Yoshua Bengio. Manifold mixup: Better representations by interpolating hidden states. In Kamalika Chaudhuri and Ruslan Salakhutdinov (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 6438–6447. PMLR, 09–15 Jun 2019. URL https://proceedings.mlr.press/v97/verma19a.html.

Liwei Wang, Lunjia Hu, Jiayuan Gu, Zhiqiang Hu, Yue Wu, Kun He, and John Hopcroft. Towards understanding learning representations: To what extent do different neural networks learn the same representation. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper/2018/file/5fc34ed307aac159a30d81181c99847e-Paper.pdf.

Alex H Williams, Erin Kunz, Simon Kornblith, and Scott Linderman. Generalized shape metrics on neural representations. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems*, 2021. URL https://openreview.net/forum?id=L9JM-pxQOl.

Matthew D. Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *Computer Vision – ECCV 2014*, pp. 818–833. Springer International Publishing, 2014a. doi: 10.1007/978-3-319-10590-1_53. URL https://doi.org/10.1007/978-3-319-10590-1_53.

Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European conference on computer vision*, pp. 818–833. Springer, 2014b.

Daquan Zhou, Bingyi Kang, Xiaojie Jin, Linjie Yang, Xiaochen Lian, Zihang Jiang, Qibin Hou, and Jiashi Feng. Deepvit: Towards deeper vision transformer. *arXiv preprint arXiv:2103.11886*, 2021.

# A EXPERIMENTAL DETAILS

## A.1 MINIBATCH CKA

In our experiments (with the exception of subsection 4.2), in order to reduce memory consumption, we use the minibatch implementation of the CKA similarity Nguyen et al. (2021; 2022). More precisely, let $\psi$ be a kernel function (we experiment with linear and RBF kernel), and $X_b \in \mathbb{R}^{m \times n_1}$ and $Y_b \in \mathbb{R}^{m \times n_2}$ be the minibatches of $m$ samples from two network layers containing $n_1$ and $n_2$ neurons respectively. We estimate the value of CKA by averaging the Hilbert-Schmidt independence criterion (HSIC), over all minibatches $b \in B$ via:

$$\mathrm{CKA}_{\mathrm{minibatch}} = \frac{\frac{1}{|B|} \sum_{b \in B} \mathrm{HSIC}_1(Q_b, Z_b)}{\sqrt{\frac{1}{|B|} \sum_{b \in B} \mathrm{HSIC}_1(Q_b, Q_b)} \sqrt{\frac{1}{|B|} \sum_{b \in B} \mathrm{HSIC}_1(Z_b, Z_b)}} \tag{4}$$

Where $Q_b = \psi(X_b, X_b)$, $Z_b = \psi(Y_b, Y_b)$, and $\mathrm{HSIC}_1$ is the unbiased estimator of HSIC Song et al. (2012), hence the value of the CKA is independent of the batch size.

## A.2 NETWORK ARCHITECTURE

In Sec. 4.1 we use a 9 layer neural network; the first 8 of these layers are convolution layers and the last layer is a fully connected layer used for classification. We use ReLU (Nair & Hinton, 2010) throughout the network. The kernel size of every convolution layer is set to $(3, 3)$ except the first two convolution layers, which have $(7, 7)$ kernels. All convolution layers follow a padding of 0 and a stride of 1. Number of kernels in each layer of the network, from the lower layers onward follows: $[16, 16, 32, 32, 32, 64, 64]$. In this network, every convolution layer is followed by batch normalization (Ioffe & Szegedy, 2015). The network we used in Sec. 4.3 to obtain Figures 5 and 6 is similar to the network we just described, except the kernel size for all layers are set to $(3, 3)$. For the **Wider networks** experiments in Sec. 4.3 we use a ResNet-34 (He et al., 2016) network, where we scale up the channels of the network to increase the width of the network (see Fig. 7).

## A.3 TRAINING DETAILS

The models in Sec. 4.1, both the generalized and memorized network, were trained for 100 epochs using AdamW (Loshchilov & Hutter, 2017) optimizer with a learning rate (LR) of $1e{-}3$ and a weight decay of $5e{-}4$. The LR is follows cosine LR scheduler (Loshchilov & Hutter, 2016) with an initial LR stated earlier.

The training of the base model (original) model in Sec. 4.3 seen in Figures 5 and 6 follows the same training procedure as of the models from Sec. 4.1, except in this setting we train the model for 200 epochs, with an initial LR of 0.01. All other models in Sec. 4.3 seen in Figures 5 and 6 (with a target CKA map to optimize) are also trained with similar training hyperparameters to that of the base model, except the followings: (1) these models are only trained for 30 epochs. (2) the objective function includes a hyperparameter $\lambda$ (see Eq. 3), which we initially set to 500 for all models and is changed dynamically following the Algo. 1 during the training by 0.8 on each iteration. (3) The cosine LR scheduler includes a warm-up step of 500 optimization steps. (4) the LR is set to $1e{-}3$ (4) The distillation loss in the objective function depends on a temperature parameter, which we set to 0.2.

The training procedure for the **Wider networks** experiments in Sec. 4.3 is similar to the previous training procedures in this section (Figures 5 and 6). Except that the *Original* models are trained for only 100 epochs and the *Optimized w.r.t Target Maps* models are trained for 15 epochs.

## A.4 CKA MAP LOSS BALANCE

Algo. 1 shows the pseudo code of the dynamical scaling of the $\lambda$ loss balance parameter seen in Eq. 3. Using the validation set accuracy as a surrogate metric for how well the network's representations are preserved, $\lambda$ is then modulated to learn maps. If the difference between the original accuracy of the network and the current validation accuracy is above a certain threshold ($\delta_{\mathrm{acc}}$) we scale down $\lambda$ to emphasize the alignment of the network output with the outputs of $f_{\theta^*}$, otherwise we scale it up to encourage finer agreement between the target and network CKA maps.